BSR Quality Checker - Technical Deep Dive

For Engineers, CTOs, and Technical Decision-Makers

The Technical Problem

Problem Space: Regulatory Document Compliance Checking

We're solving a **structured information extraction + rule-based validation problem** applied to unstructured regulatory documents.

Input: Heterogeneous PDF documents (fire strategies, structural calcs, architectural drawings, specs)

- Scanned images vs native PDFs
- Tables, diagrams, dense technical text
- 50-500 pages per document set
- No standardized format or schema

Required Output: Structured compliance assessment

- Binary/ternary status per requirement (meets/partial/does_not_meet)
- Evidence provenance (document name + page number + quote)
- Gap identification (what's missing)
- Actionable remediation steps

Constraints:

- Must check 55+ specific regulatory requirements
- Must be deterministic where possible (same doc → same result)
- Must provide audit trail (traceable to source)

- Must run in <5 minutes for typical document set
- Must handle UK Building Safety Regulator (BSR) domain complexity

Why This Is Hard

Challenge 1: Unstructured → Structured Transformation

```
Unstructured PDF text:
"The building adopts a stay-put evacuation strategy in line with BS 9991,
with compartmentation designed to resist fire spread for 60 minutes..."

→ Must extract:
- evacuation_strategy: "stay_put"
- standard_reference: "BS 9991"
- compartmentation_rating: "60 minutes"
- compliance_status: meets/does_not_meet for REQ-FS-005
```

Traditional NLP/NER struggles because:

- Domain-specific terminology (e.g., "compartmentation", "HRB", "Gateway 2")
- Implicit requirements (building height 18m triggers different rules)
- Negations and conditionals ("except where...", "provided that...")
- Cross-document references (fire strategy references structural calcs)

Challenge 2: Deterministic Rules + Probabilistic AI

- Some requirements are binary: "Fire strategy must be present" (deterministic)
- Others need interpretation: "Evacuation strategy must be appropriate for building type" (requires context)
- Need hybrid approach: rules engine + LLM enrichment

Challenge 3: Evidence Provenance LLMs can answer "Is requirement met?" but struggle with:

- "Which document contains the evidence?" (hallucinate filenames)
- "What page number?" (hallucinate page refs)
- "Exact quote?" (paraphrase instead of quoting)

We need deterministic document/page linking, not just semantic understanding.

Challenge 4: Compliance Domain Complexity

- 55+ interconnected requirements
- Conditional logic (IF height >= 18m THEN HRB rules apply)
- London-specific overlays (Building Act 1984)
- Building type variations (residential vs mixed-use)
- Evolving regulations (BSR guidance updates quarterly)

Why Existing Solutions Don't Work

Manual Consultant Review:

- Takes 5-20 hours per project
- Error-prone (humans miss requirements)
- Not scalable (consultant bottleneck)
- No standardization (varies by reviewer)

Generic Document AI (ChatGPT/Claude):

- No domain model (doesn't know BSR requirements)
- Hallucinates evidence (makes up page numbers)
- No structured output (markdown blob instead of JSON)
- No deterministic rules (probabilistic only)
- Requires prompt engineering per use case

Rule-Based Systems (Traditional NLP):

- Brittle regex patterns fail on natural language variation
- Can't handle "Does this fire strategy adequately address evacuation?"
- Require extensive training data for each requirement
- Break when document format changes

Our Approach: Hybrid Rules Engine + LLM Enrichment Combine strengths of both:

- **Deterministic rules** for binary checks ("fire strategy present?")
- **LLM** for semantic understanding ("is evacuation strategy appropriate?")

- **Structured prompting** to force provenance and reasoning
- **Post-processing** to validate evidence and prevent hallucinations

---

System Architecture

High-Level Data Flow

```
|  - Reasoning generation                 |
|  - Gap identification                   |
└─────────────┬─────────────────────────────┘
              |
              v
┌─────────────────────────────────────────┐
|    Report Generation Pipeline           |
|  7. Generate compliance matrix (Excel)  |
|  8. Generate client gap analysis (PDF)  |
|  9. Generate action plan (PDF)          |
└─────────────┬─────────────────────────────┘
              |
              v
┌─────────────────────────────────────────┐
|         Database (PostgreSQL)           |
|  - Save assessment results              |
|  - Link to client/pack                  |
|  - Store version history                |
└─────────────────────────────────────────┘
```

Component Breakdown

1. Frontend (React + TypeScript + Vite)

Tech Stack:

- React 18 (functional components + hooks)
- TypeScript (strict mode)
- Vite for dev server + HMR
- No state management library (useState/useEffect sufficient for now)
- Fetch API for HTTP (no axios)

Key Files:

```
packages/frontend/src/
├── pages/
|   ├── QuickAssess.tsx          # Main assessment workflow
|   ├── ClientDetail.tsx         # Client/pack management
|   └── PackDetail.tsx           # Individual pack view
├── components/
|   ├── FileUpload.tsx           # Drag-drop file upload
|   ├── SimpleResultsView.tsx    # Assessment results modal
|   └── ResultsDashboard.tsx     # Stats visualization
```

```
├── services/
|   └── exportService.ts         # API calls for PDF/Excel export
└── types/
    └── assessment.ts            # TypeScript interfaces
```

State Management Pattern:

```
// Local component state (useState)
const [assessment, setAssessment] = useState<QuickAssessment | null>(null);
const [assessing, setAssessing] = useState(false);

// API call
const res = await fetch('/api/assess', {
  method: 'POST',
  body: formData
});
const data = await res.json();
setAssessment(data);
```

File Upload Handling:

- `FormData` for multipart/form-data
- Client-side file size validation
- Progress tracking via state updates from backend events

2. Backend (Node.js + Express + TypeScript)

Tech Stack:

- Node.js v24
- Express.js (REST API)
- TypeScript with tsx for dev, compiled for prod
- Prisma ORM (type-safe database access)
- Puppeteer (headless Chrome for PDF generation)
- ExcelJS (Excel spreadsheet generation)

Project Structure:

```
packages/backend/src/
├── index.ts                    # Express app setup
├── routes/
```

```
|   ├── assess.ts               # POST /api/assess (main assessment endpoint)
|   ├── export.ts               # PDF/Excel generation endpoints
|   ├── clients.ts              # Client CRUD
|   └── packs.ts                # Pack CRUD
├── services/
|   ├── assessment.ts           # Core assessment logic
|   ├── compliance-matrix.ts    # Matrix data transformation
|   ├── excel-export.ts         # Excel generation
|   └── rag.ts                  # Document chunking/retrieval (planned)
├── prisma/
|   └── schema.prisma           # Database schema
└── types/
    └── assessment.ts           # Shared types
```

Key Middleware:

```
app.use(express.json({ limit: '10mb' }));  // Increased for large assessments
app.use(express.static('public'));          // Serve test pages
app.use('/api/assess', assessRouter);
app.use('/api/packs', packsRouter);
```

Error Handling Pattern:

```
try {
  // Assessment logic
  const assessment = await runAssessment(documents);
  res.json(assessment);
} catch (error) {
  console.error('Assessment failed:', error);
  res.status(500).json({
    error: error instanceof Error ? error.message : 'Unknown error'
  });
}
```

3. Database Layer (Prisma + PostgreSQL)

Why Prisma?

- Type-safe queries (generated TypeScript types from schema)
- Migrations built-in
- Good DX (autocomplete in IDE)
- Easy to switch between SQLite (dev) and PostgreSQL (prod)

Schema Design:

```
model Client {
  id           String   @id @default(uuid())
  name         String
  company      String?
  contactEmail String?
  createdAt    DateTime @default(now())
  packs        Pack[]
}

model Pack {
  id           String        @id @default(uuid())
  name         String
  clientId     String?
  client       Client?       @relation(fields: [clientId], references: [id])
  versions     PackVersion[]
  createdAt    DateTime      @default(now())
}

model PackVersion {
  id               String      @id @default(uuid())
  packId           String
  pack             Pack        @relation(fields: [packId], references: [id])
  versionNumber    Int
  matrixAssessment String?     // JSON blob of assessment results
  createdAt        DateTime    @default(now())
  documents        Document[]

  @@unique([packId, versionNumber])
}

model Document {
  id            String       @id @default(uuid())
  packVersionId String?
  packVersion   PackVersion? @relation(fields: [packVersionId], references: [id])
  filename      String
  filepath      String
  docType       String?      // "fire_strategy", "structural_calcs", etc.
  createdAt     DateTime     @default(now())
}
```

Key Design Decisions:

1. **Why JSON blob for matrixAssessment?**
    - Assessment results are complex nested objects (~50KB per assessment)

- Schema is still evolving (adding new fields frequently)
- Don't need to query inside assessment data (only retrieve full blob)
- Trade-off: Can't filter by specific requirement status in SQL

Alternative considered: Separate `AssessmentResult` table per requirement

- ✅ Queryable, normalized
- ❌ 55 rows per assessment = lots of joins
- ❌ Schema changes require migrations
- Decision: JSON blob for now, migrate to normalized if needed for analytics

2. **Why PackVersion instead of updating Pack?**
   - Enables version history (iterate on same project)
   - Can compare v1 vs v2 assessments
   - Immutable audit trail
   - Supports "run new assessment on updated documents" workflow
3. **Client → Pack → PackVersion hierarchy**
   - Maps to consultancy workflow (client has multiple projects)
   - Pack = project/building
   - PackVersion = submission attempt (Gateway 2 v1, v2 after revisions)

Prisma Usage Pattern:

```
// Type-safe query with relations
const client = await prisma.client.findUnique({
  where: { id: clientId },
  include: {
    packs: {
      include: {
        versions: {
          orderBy: { versionNumber: 'desc' },
          take: 1
        }
      }
    }
  }
});
```

```
// Generated TypeScript type:
// client: Client & { packs: (Pack & { versions: PackVersion[] })[] }
```

4. Assessment Engine (Core Logic)

File: `packages/backend/src/services/assessment.ts`

High-Level Flow:

```
async function runAssessment(documents: Document[], context: BuildingContext) {
  // 1. Extract text from PDFs
  const extractedDocs = await Promise.all(
    documents.map(doc => extractTextFromPDF(doc))
  );

  // 2. Build document index (for evidence linking)
  const docIndex = buildDocumentIndex(extractedDocs);

  // 3. Run 55 compliance rules
  const results = await Promise.all(
    COMPLIANCE_RULES.map(rule =>
      checkRequirement(rule, extractedDocs, docIndex, context)
    )
  );

  // 4. Aggregate and return
  return {
    results,
    summary: calculateSummary(results),
    context
  };
}
```

Compliance Rule Structure:

```
interface ComplianceRule {
  id: string;                    // "REQ-FS-005"
  category: string;              // "Fire Safety"
  title: string;                 // "Evacuation Strategy"

  // Deterministic check (optional)
  condition?: (docs: Document[], context: BuildingContext) => boolean;

  // LLM-based check (if condition not sufficient)
  llmPrompt?: (docs: Document[], context: BuildingContext) => string;
```

```
  // Evidence extraction strategy
  evidenceStrategy: 'keyword' | 'llm' | 'both';
}

// Example rule
const RULE_FS_005: ComplianceRule = {
  id: 'REQ-FS-005',
  category: 'Fire Safety',
  title: 'Evacuation strategy must be specified',

  // Deterministic: Check if fire strategy doc exists
  condition: (docs, context) => {
    return docs.some(d => d.type === 'fire_strategy');
  },

  // LLM: Extract evacuation strategy type
  llmPrompt: (docs, context) => `
    Review the fire strategy document and identify:
    1. Is an evacuation strategy explicitly specified? (stay put / simultaneous /
    2. What page is this mentioned on?
    3. Quote the exact text.

    Return JSON:
    {
      "status": "meets" | "partial" | "does_not_meet",
      "evacuation_type": "stay_put" | "simultaneous" | "phased" | null,
      "evidence_doc": "filename.pdf",
      "evidence_page": 12,
      "evidence_quote": "exact quote here",
      "reasoning": "why this meets/doesn't meet requirement"
    }
  `,

  evidenceStrategy: 'llm'
};
```

LLM Integration Pattern:

```
async function checkWithLLM(
  rule: ComplianceRule,
  docs: Document[],
  context: BuildingContext
): Promise<AssessmentResult> {
  const prompt = rule.llmPrompt(docs, context);

  // Call Claude API
```

```
  const response = await anthropic.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 2000,
    temperature: 0,  // Deterministic
    messages: [{
      role: 'user',
      content: [
        {
          type: 'text',
          text: `Here are the documents:\n\n${docs.map(formatDoc).join('\n\n')}`
        },
        {
          type: 'text',
          text: prompt
        }
      ]
    }]
  });

  // Parse structured JSON response
  const result = JSON.parse(response.content[0].text);

  // Validate evidence (prevent hallucinations)
  const validated = validateEvidence(result, docs);

  return validated;
}
```

Evidence Validation (Anti-Hallucination):

```
function validateEvidence(
  result: LLMResponse,
  docs: Document[]
): AssessmentResult {
  // 1. Check document exists
  const doc = docs.find(d => d.filename === result.evidence_doc);
  if (!doc) {
    console.warn(`LLM hallucinated document: ${result.evidence_doc}`);
    result.evidence_doc = null;  // Clear invalid evidence
  }

  // 2. Check page number in range
  if (result.evidence_page > doc.totalPages) {
    console.warn(`LLM hallucinated page: ${result.evidence_page} > ${doc.totalPa
    result.evidence_page = null;
  }
```

```
  // 3. Verify quote exists in document (fuzzy match)
  if (result.evidence_quote) {
    const pageText = doc.pages[result.evidence_page - 1];
    const similarity = stringSimilarity(result.evidence_quote, pageText);
    if (similarity < 0.7) {
      console.warn(`LLM quote not found in document (similarity: ${similarity})`
      result.evidence_quote = null;  // Quote doesn't match source
    }
  }

  return result;
}
```

5. Report Generation Pipeline

Three Output Formats:

1. **Client Gap Analysis (PDF)**
   - Puppeteer headless Chrome
   - HTML template → PDF rendering
   - Focuses on missing information client needs to provide
2. **Consultant Action Plan (PDF)**
   - Same tech stack (Puppeteer)
   - Internal working document
   - Prioritized issues with owner assignments
3. **Compliance Matrix (Excel)**
   - ExcelJS library
   - Programmatic spreadsheet generation
   - Color-coded cells, frozen headers, auto-filters

PDF Generation (Puppeteer):

```
async function generatePDF(assessment: Assessment): Promise<Buffer> {
  // 1. Build HTML from template
  const html = generateHTML(assessment);

  // 2. Launch headless Chrome
```

```
  const browser = await puppeteer.launch({
    args: ['--no-sandbox', '--disable-setuid-sandbox'],  // Railway compat
    headless: true
  });

  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'networkidle0' });

  // 3. Render PDF
  const pdf = await page.pdf({
    format: 'A4',
    margin: { top: '0.75in', bottom: '0.75in', left: '0.75in', right: '0.75in' }
    printBackground: true  // Include CSS backgrounds/colors
  });

  await browser.close();
  return pdf;
}
```

Excel Generation (ExcelJS):

```
async function generateComplianceMatrixExcel(
  matrix: ComplianceMatrix
): Promise<Buffer> {
  const workbook = new ExcelJS.Workbook();
  const worksheet = workbook.addWorksheet('Compliance Matrix', {
    views: [{ state: 'frozen', xSplit: 0, ySplit: 1 }]  // Freeze header
  });

  // Add summary section
  worksheet.addRow(['BSR COMPLIANCE MATRIX']);
  worksheet.addRow(['Project:', matrix.projectName]);
  worksheet.addRow(['Generated:', new Date().toLocaleDateString()]);
  worksheet.addRow(['Generated by:', '🤖 AI-Powered Analysis']);

  // Add data table with color coding
  const headerRow = worksheet.addRow([
    'ID', 'Requirement', 'Category', 'Status', 'Priority',
    'Evidence Document', 'Page', 'Action', 'Owner', 'Notes'
  ]);

  headerRow.font = { bold: true, color: { argb: 'FFFFFFFF' } };
  headerRow.fill = {
    type: 'pattern',
    pattern: 'solid',
    fgColor: { argb: 'FF1E40AF' }  // Navy blue
  };
```

```
  // Add data rows with conditional formatting
  matrix.rows.forEach(row => {
    const excelRow = worksheet.addRow({
      id: row.requirementId,
      requirement: row.requirement,
      status: row.status,
      // ... etc
    });

    // Color code status cell
    const statusCell = excelRow.getCell('status');
    if (row.status === 'Met') {
      statusCell.fill = {
        type: 'pattern',
        pattern: 'solid',
        fgColor: { argb: 'FFD4EDDA' }  // Green
      };
    }
    // ... Yellow for Partial, Red for Not Met
  });

  // Export to buffer
  const buffer = await workbook.xlsx.writeBuffer();
  return Buffer.from(buffer);
}
```

AI/LLM Integration Details

Model Selection: Claude Sonnet 4.5

Why Claude over GPT-4?

- Better at following structured output formats (JSON reliability)
- Longer context window (200K tokens) for large document sets
- Better instruction following for compliance-style prompts
- Lower hallucination rate on factual extraction tasks
- Native support for PDF/image inputs (future: analyze drawings directly)

Model Config:

```
const response = await anthropic.messages.create({
  model: 'claude-sonnet-4-20250514',  // Latest Sonnet 4.5
  max_tokens: 2000,                    // Enough for detailed responses
  temperature: 0,                      // Deterministic (same input → same output
  messages: [...]
});
```

Temperature = 0 Rationale:

- Compliance assessment should be deterministic
- Same documents should produce same results
- No creativity needed (not generating creative content)
- Reduces variability across runs

Prompt Engineering Strategy

Structured Output Enforcement:

```
const prompt = `
You are a building safety compliance expert reviewing Gateway 2 submission docume

REQUIREMENT: ${rule.title}
CRITERIA: ${rule.description}

Review the provided documents and assess compliance.

IMPORTANT: Respond ONLY with valid JSON in this exact format:
{
  "status": "meets" | "partial" | "does_not_meet",
  "evidence_document": "exact_filename.pdf" | null,
  "evidence_page": <page_number> | null,
  "evidence_quote": "exact quote from document" | null,
  "reasoning": "2-3 sentence explanation of your assessment",
  "gaps_identified": ["gap1", "gap2"],
  "actions_required": [{
    "action": "what needs to be done",
    "owner": "Fire Engineer" | "Structural Engineer" | "Architect" | "Client",
    "effort": "S" | "M" | "L"
  }]
}

Do not include any text outside the JSON object.
`;
```

Why this works:

- Clear role definition ("building safety compliance expert")
- Explicit schema with type constraints
- "IMPORTANT" directive for emphasis
- Example format prevents misinterpretation
- "Do not include any text outside JSON" prevents markdown wrapping

Prompt Testing Results:

- Without "respond ONLY with valid JSON": ~20% invalid JSON responses (wrapped in markdown, extra text)
- With explicit schema + "ONLY": <2% invalid responses
- Temperature 0 vs 0.7: 0 reduces hallucinated page numbers from ~15% to ~5%

Context Window Management

Problem: Large document sets exceed context window

- Fire strategy: 50-150 pages
- Structural calcs: 100-300 pages
- Specs: 200+ pages
- Total: Can easily exceed 200K tokens

Current Strategy (Naive - Works for MVP):

```
// Concatenate all document text
const allText = documents.map(d => d.text).join('\n\n');

// Truncate if too large
const MAX_CHARS = 400000;  // ~100K tokens
if (allText.length > MAX_CHARS) {
  allText = allText.substring(0, MAX_CHARS) + '\n\n[TRUNCATED]';
}
```

Limitations:

- Loses information beyond truncation point
- No intelligent selection of relevant sections

- Wastes tokens on irrelevant content

Future Strategy (RAG - Planned):

```
// 1. Chunk documents into semantic sections
const chunks = documents.flatMap(d => chunkDocument(d, { size: 1000, overlap: 200

// 2. Embed chunks (vector embeddings)
const embeddings = await embed(chunks);

// 3. Store in vector DB
await vectorDB.upsert(embeddings);

// 4. For each requirement, retrieve relevant chunks
const relevantChunks = await vectorDB.query(
  embed(rule.title + ' ' + rule.description),
  { topK: 10 }
);

// 5. Pass only relevant chunks to LLM
const response = await checkWithLLM(rule, relevantChunks, context);
```

RAG Implementation Plan:

- Vector DB: Pinecone or Weaviate (managed) or pg_vector (self-hosted)
- Embedding model: OpenAI text-embedding-3-small (cheap, fast)
- Chunk strategy: Sliding window (1000 chars, 200 overlap)
- Retrieval: Hybrid (vector similarity + keyword matching)

Expected Improvements:

- Reduce token usage by 80% (only send relevant sections)
- Handle unlimited document size (no truncation)
- Improve accuracy (more context per requirement)
- Enable citation linking (chunk → document → page)

Hallucination Mitigation

Problem: LLMs hallucinate evidence

- Make up page numbers that don't exist

- Reference documents not in the set
- Paraphrase instead of exact quotes
- Invent compliance status

Mitigation Strategies:

1. **Post-Processing Validation (Currently Implemented)**

```
// Check document exists
if (!docs.find(d => d.filename === result.evidence_doc)) {
  result.evidence_doc = null;  // Clear hallucinated doc
}

// Check page in range
if (result.evidence_page > doc.totalPages) {
  result.evidence_page = null;  // Clear hallucinated page
}

// Fuzzy match quote
if (stringSimilarity(result.quote, doc.pages[page]) < 0.7) {
  result.evidence_quote = null;  // Quote doesn't match
}
```

2. **Structured Output Constraints (Prompt Engineering)**

```
"evidence_document": "exact_filename.pdf" | null,

IMPORTANT: evidence_document must be EXACTLY one of these filenames:
- fire_strategy_v2.pdf
- structural_calcs_2024.pdf
- ...

If evidence is not found, return null. Do NOT guess or infer.
```

3. **Few-Shot Examples (Planned)**

```
const prompt = `
Here are examples of correct assessments:

Example 1:
Requirement: Fire strategy must specify evacuation strategy
Document: fire_strategy.pdf, Page 12
Quote: "The building adopts a stay-put evacuation strategy..."
Status: meets
```

```
Example 2:
Requirement: Structural calcs must reference Eurocodes
No evidence found
Status: does_not_meet

Now assess this requirement:
...
`;
```

4. **Confidence Scoring (Future)**

```
{
  "status": "meets",
  "confidence": 0.85,  // LLM self-reports confidence
  "evidence_quote": "...",
  "quote_similarity": 0.92  // Post-processing validation score
}
```

   Flag low-confidence results for human review.

Performance & Scalability

Current Performance (MVP)

Assessment Speed:

- Single document (50 pages): ~30 seconds
- Typical set (5 docs, 200 pages): ~2-3 minutes
- Large set (10 docs, 500 pages): ~5-7 minutes

Bottlenecks:

1. **LLM API calls** (55 sequential calls to Claude)
   - Each rule check: ~2-5 seconds
   - Serial execution: 55 * 3s = ~165s baseline
2. **PDF text extraction**

- Using pdf-parse library
- ~500ms per 100-page PDF
- Not parallelized currently

3. **Report generation (Puppeteer)**
   - HTML → PDF rendering: ~3-5 seconds per report
   - Headless Chrome startup: ~1-2 seconds

Optimization Strategies

1. Parallel LLM Calls

```
// Before (serial)
for (const rule of RULES) {
  results.push(await checkWithLLM(rule, docs, context));
}

// After (parallel)
const results = await Promise.all(
  RULES.map(rule => checkWithLLM(rule, docs, context))
);
```

Expected speedup: 55 serial → 55 parallel = **165s → 5s** (limited by API rate limits)

2. Anthropic Rate Limits:

- Tier 1: 50 requests/minute, 40K tokens/minute
- With 55 parallel calls, we hit request limit
- Solution: Batch in groups of 50

```
const batchSize = 50;
for (let i = 0; i < RULES.length; i += batchSize) {
  const batch = RULES.slice(i, i + batchSize);
  const batchResults = await Promise.all(
    batch.map(rule => checkWithLLM(rule, docs, context))
  );
  results.push(...batchResults);
}
```

3. Caching (Redis - Planned)

```
// Cache LLM responses by (document hash + rule ID)
const cacheKey = `assessment:${docHash}:${rule.id}`;
const cached = await redis.get(cacheKey);
if (cached) return JSON.parse(cached);

const result = await checkWithLLM(rule, docs, context);
await redis.set(cacheKey, JSON.stringify(result), 'EX', 86400);  // 24h TTL
return result;
```

Benefits:

- Re-running same documents: instant (cache hit)
- Iterative workflow (client updates one doc): only re-check affected rules
- Reduce API costs (no redundant LLM calls)

4. Background Processing (Bull Queue - Planned)

```
// Frontend: Start assessment, return job ID
POST /api/assess
→ { job_id: "uuid-1234" }

// Backend: Queue assessment job
await assessmentQueue.add('assess', { documents, context });

// Worker: Process jobs asynchronously
assessmentQueue.process('assess', async (job) => {
  const result = await runAssessment(job.data.documents, job.data.context);
  await saveResult(job.data.userId, result);
  await sendNotification(job.data.userId, 'Assessment complete');
});

// Frontend: Poll for completion
GET /api/assess/status/:jobId
→ { status: "processing" | "completed", result?: Assessment }
```

Benefits:

- Non-blocking API (immediate response)
- Handle long-running assessments (10+ minutes)
- Scale workers independently
- Retry failed jobs

Scaling Considerations

Current Architecture (Single Server):

```
Railway Deployment (1 instance)
├── Express API
├── Puppeteer (headless Chrome)
├── Prisma (PostgreSQL connection)
└── Anthropic API client
```

Limitations:

- Single point of failure
- Puppeteer memory-intensive (500MB+ per instance)
- No horizontal scaling (Railway auto-scale coming but not configured)

Scaling Path (Multi-Instance):

```
Load Balancer (Railway provided)
├── API Server 1 (stateless)
├── API Server 2 (stateless)
└── API Server 3 (stateless)
       |
       ├→ Redis (session/cache)
       ├→ PostgreSQL (data)
       ├→ Bull Queue (job queue)
       └→ Worker Pool
           ├── Assessment Worker 1
           ├── Assessment Worker 2
           └── PDF Generator Worker
```

What Needs to Change:

1. **Stateless API servers** (currently stateful - stores temp files)
    - Move uploaded files to S3/R2
    - Use job queue for assessments
2. **Separate PDF generation** (Puppeteer heavy)
    - Dedicated worker instances
    - Or use external service (Browserless.io)

3. **Connection pooling** (Prisma)
    - Configure `connection_limit` for PostgreSQL
    - Use PgBouncer for connection pooling

Expected Capacity:

- Current: ~10 concurrent assessments (limited by Puppeteer memory)
- Multi-instance: ~100 concurrent (10 workers * 10 assessments each)
- With caching: ~1000 assessments/hour (80% cache hit rate)

Deployment & DevOps

Current Stack

Hosting: Railway (PaaS)

- Automatic deployment from GitHub push
- PostgreSQL managed database
- Environment variable management
- Zero-config TLS/SSL

Why Railway?

- ✅ Simple deployment (git push → live)
- ✅ Managed PostgreSQL (no DB admin)
- ✅ Built-in CI/CD (no GitHub Actions needed)
- ✅ Reasonable pricing ($5-20/month for MVP)
- ❌ Limited control (can't customize infra)
- ❌ Vendor lock-in (migration path unclear)

Environment Variables:

```
# Backend
DATABASE_URL="postgresql://user:pass@host:5432/db"
ANTHROPIC_API_KEY="sk-ant-..."
```

```
NODE_ENV="production"

# Frontend
VITE_API_URL="https://api.bsr-checker.com"
```

Build & Deploy Process

```
# Local development
cd packages/backend && npm run dev    # tsx watch (hot reload)
cd packages/frontend && npm run dev   # vite dev server

# Pre-push checks (Git hook)
npx tsc --noEmit                      # TypeScript type check
# → Prevents pushing code that doesn't compile

# Git push → Railway deploy
git push origin main
# → Railway detects push
# → Runs npm install
# → Runs npm run build (TypeScript → JavaScript)
# → Starts server with npm start
# → Zero-downtime deployment (new instance → drain old instance)
```

Database Migrations

Prisma Migrate:

```
# Create migration
npx prisma migrate dev --name add_client_model

# Apply to production
npx prisma migrate deploy
```

Migration Strategy:

- Dev: SQLite (`file:./dev.db`) - Fast iteration, no setup
- Prod: PostgreSQL on Railway - Managed, reliable
- Same Prisma schema works for both (provider switch)

Schema Evolution:

```
// Add new field (safe - non-breaking)
model Pack {
  id String @id
  name String
  status String @default("draft")  // ← New field with default
}

// Remove field (BREAKING - requires data migration)
// Step 1: Make optional
model Pack {
  oldField String?  // ← Mark nullable
}
// Step 2: Deploy, migrate data
// Step 3: Remove field
```

Monitoring & Observability (Planned)

Current State:

- Console.log only
- No error tracking
- No performance monitoring
- No uptime alerts

Planned Additions:

1. **Error Tracking: Sentry**

```
import * as Sentry from '@sentry/node';

Sentry.init({ dsn: process.env.SENTRY_DSN });

app.use(Sentry.Handlers.errorHandler());
```

2. **Logging: Pino (structured logging)**

```
import pino from 'pino';
const logger = pino();

logger.info({ assessmentId, duration }, 'Assessment completed');
logger.error({ error, stack }, 'Assessment failed');
```

3. **Metrics: Prometheus + Grafana**
    - Request rate, latency, error rate
    - LLM API call success/failure rate
    - Assessment duration histogram
    - Active assessments gauge
4. **Uptime Monitoring: Better Uptime**
    - Health check endpoint: `GET /health`
    - Ping every 1 minute
    - Alert on downtime > 5 minutes

Security Considerations

Data Sensitivity

What We Handle:

- Building plans (potentially sensitive)
- Client information (names, companies, emails)
- Structural/fire safety details (could be security concern for high-profile buildings)

Threat Model:

- Data breach: Client documents exposed
- Unauthorized access: Competitor views assessments
- Data leakage: LLM provider (Anthropic) sees confidential plans

Current Security Measures

1. HTTPS Only

- Railway provides TLS termination
- All traffic encrypted in transit

2. Database Access Control

- PostgreSQL credentials in env vars (not committed)
- Connection from Railway backend only (not public)

3. No Authentication (MVP Risk)

- ⚠️ Anyone can upload documents and run assessments
- ⚠️ No user accounts or access control
- ⚠️ No client data isolation

Acceptable for MVP because:

- No billing (no monetary risk)
- No sensitive production data yet
- Testing with internal/demo documents only

4. LLM Data Privacy (Anthropic)

- Anthropic's data policy: Does not train on API inputs
- Enterprise plan offers SOC 2 compliance
- Documents sent to Anthropic for processing (not stored long-term)

Required for Production

1. Authentication (Clerk - Planned)

```
import { ClerkExpressWithAuth } from '@clerk/clerk-sdk-node';

app.use(ClerkExpressWithAuth());

app.post('/api/assess', (req, res) => {
  const userId = req.auth.userId;  // From Clerk JWT
  if (!userId) return res.status(401).json({ error: 'Unauthorized' });

  // Run assessment for this user
});
```

2. Authorization (Row-Level Security)

```
-- PostgreSQL RLS
ALTER TABLE packs ENABLE ROW LEVEL SECURITY;

CREATE POLICY user_packs ON packs
  FOR ALL
  USING (user_id = current_setting('app.user_id')::uuid);
```

3. Rate Limiting (express-rate-limit)

```
import rateLimit from 'express-rate-limit';

const assessmentLimiter = rateLimit({
  windowMs: 60 * 60 * 1000,  // 1 hour
  max: 10,                   // 10 assessments per hour per IP
  message: 'Too many assessments, please try again later'
});

app.post('/api/assess', assessmentLimiter, async (req, res) => {
  // ...
});
```

4. Input Validation (Zod)

```
import { z } from 'zod';

const AssessmentSchema = z.object({
  buildingType: z.enum(['residential', 'commercial', 'mixed']),
  isHRB: z.boolean(),
  isLondon: z.boolean(),
  documents: z.array(z.object({
    filename: z.string().max(255),
    size: z.number().max(50 * 1024 * 1024)  // 50MB max
  }))
});

app.post('/api/assess', async (req, res) => {
  const parsed = AssessmentSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ error: parsed.error });
  }
  // ...
});
```

5. File Upload Security

```
import multer from 'multer';

const upload = multer({
  limits: { fileSize: 50 * 1024 * 1024 },  // 50MB max
  fileFilter: (req, file, cb) => {
    // Only allow PDFs
    if (file.mimetype !== 'application/pdf') {
      cb(new Error('Only PDF files allowed'));
    } else {
      cb(null, true);
    }
  }
});

app.post('/api/assess', upload.array('documents'), async (req, res) => {
  // Validate PDF integrity (not just mimetype)
  for (const file of req.files) {
    const isValidPDF = await validatePDFStructure(file.path);
    if (!isValidPDF) {
      return res.status(400).json({ error: 'Invalid PDF file' });
    }
  }
  // ...
});
```

Testing Strategy

Current State (Minimal)

Unit Tests: None **Integration Tests:** None **E2E Tests:** None **Manual Testing:** Yes (via UI + test-download.html)

Why No Tests Yet?

- MVP rapid iteration (requirements changing frequently)
- Exploratory phase (figuring out what works)
- Small codebase (can manually verify)

Technical Debt Acknowledged:

- Need tests before refactoring assessment logic

- Need tests before adding more engineers
- Need tests for CI/CD confidence

Planned Testing Approach

1. Unit Tests (Vitest)

```
// packages/backend/src/services/compliance-matrix.test.ts
import { describe, it, expect } from 'vitest';
import { generateComplianceMatrix } from './compliance-matrix';

describe('generateComplianceMatrix', () => {
  it('maps assessment status correctly', () => {
    const assessment = {
      results: [
        { matrix_id: 'REQ-001', status: 'meets', matrix_title: 'Test' },
        { matrix_id: 'REQ-002', status: 'does_not_meet', matrix_title: 'Test2' }
      ]
    };

    const matrix = generateComplianceMatrix(assessment);

    expect(matrix.met).toBe(1);
    expect(matrix.notMet).toBe(1);
    expect(matrix.complianceRate).toBe(50);
  });

  it('determines priority from triage data', () => {
    const result = {
      triage: { urgency: 'CRITICAL_BLOCKER' }
    };

    const priority = determinePriority(result);

    expect(priority).toBe('Critical');
  });
});
```

2. Integration Tests (Supertest)

```
// packages/backend/src/routes/assess.test.ts
import request from 'supertest';
import { app } from '../index';

describe('POST /api/assess', () => {
```

```
  it('returns assessment results for valid documents', async () => {
    const response = await request(app)
      .post('/api/assess')
      .attach('documents', './fixtures/fire_strategy.pdf')
      .field('buildingType', 'residential')
      .field('isHRB', 'true');

    expect(response.status).toBe(200);
    expect(response.body).toHaveProperty('results');
    expect(response.body.results).toBeInstanceOf(Array);
  });

  it('rejects non-PDF files', async () => {
    const response = await request(app)
      .post('/api/assess')
      .attach('documents', './fixtures/test.txt');

    expect(response.status).toBe(400);
    expect(response.body.error).toContain('PDF');
  });
});
```

3. E2E Tests (Playwright)

```
// packages/frontend/e2e/quick-assess.spec.ts
import { test, expect } from '@playwright/test';

test('complete assessment workflow', async ({ page }) => {
  await page.goto('http://localhost:5173');

  // Navigate to Quick Assess
  await page.click('text=Quick Assess');

  // Upload document
  const fileInput = page.locator('input[type="file"]');
  await fileInput.setInputFiles('./fixtures/fire_strategy.pdf');

  // Run assessment
  await page.click('text=Run Full Assessment');

  // Wait for results
  await expect(page.locator('text=Assessment Complete')).toBeVisible({ timeout: 1

  // Download reports
  await page.click('text=Download Reports');

  // Verify downloads
```

```
  const downloads = await page.waitForEvent('download');
  expect(downloads.suggestedFilename()).toContain('client-gap-analysis');
});
```

4. LLM Output Validation Tests

```
// Snapshot testing for LLM prompts + responses
describe('LLM assessment for evacuation strategy', () => {
  it('produces consistent results', async () => {
    const docs = loadFixture('fire_strategy_stay_put.pdf');
    const result = await checkWithLLM(RULE_FS_005, docs, context);

    // Snapshot test (compare to previous output)
    expect(result).toMatchSnapshot({
      // Ignore timestamps, but check structure
      generated_at: expect.any(String)
    });
  });

  it('handles missing evidence gracefully', async () => {
    const docs = loadFixture('fire_strategy_no_evacuation.pdf');
    const result = await checkWithLLM(RULE_FS_005, docs, context);

    expect(result.status).toBe('does_not_meet');
    expect(result.gaps_identified).toContain('evacuation strategy');
  });
});
```

Test Coverage Goals:

- Critical path (assessment flow): 90%+ coverage
- Utility functions (matrix generation, Excel export): 80%+ coverage
- UI components: 60%+ coverage (lower priority)

Known Issues & Technical Debt

1. No RAG (Context Window Limitation)

Problem: Concatenate all docs → truncate if >100K tokens **Impact:** Large document sets lose information **Fix:** Implement vector DB + semantic chunking

(RAG) **Timeline:** Q2 2026 **Workaround:** Works for typical submissions (<200 pages total)

2. Serial LLM Calls (Slow)

Problem: 55 sequential API calls → ~3 minute baseline **Impact:** Slow assessments hurt UX **Fix:** Parallel batching (50 concurrent calls) **Timeline:** Q1 2026 **Workaround:** Show progress spinner with "Running Phase 2..." status

3. No Caching (Redundant API Calls)

Problem: Re-running same docs → redundant LLM calls **Impact:** Wasted API costs, slow iteration **Fix:** Redis cache keyed by (doc hash + rule ID) **Timeline:** Q1 2026 **Workaround:** Manual testing on different docs each time

4. JSON Blob for Assessment (Not Queryable)

Problem: `matrixAssessment` stored as JSON string in PostgreSQL **Impact:** Can't query "Show me all assessments with critical blockers" **Fix:** Normalize to `AssessmentResult` table **Timeline:** Q2 2026 (if analytics needed) **Workaround:** Retrieve full assessment, filter in application layer

5. No Authentication (Open Access)

Problem: Anyone can use the tool **Impact:** Security risk, no usage tracking, no billing **Fix:** Add Clerk authentication **Timeline:** Q1 2026 (before public launch) **Workaround:** Internal use only, no public URL

6. Puppeteer Memory Leak

Problem: Headless Chrome instances sometimes don't close **Impact:** Memory usage grows over time → Railway instance OOM **Fix:** Timeout on page.pdf() + explicit browser.close() in finally block **Timeline:** Q1 2026 **Workaround:** Railway auto-restarts on OOM

7. No Job Queue (Blocking Requests)

Problem: Assessment runs synchronously in HTTP request **Impact:** 5-minute request timeout risk, no concurrency control **Fix:** Bull queue + worker processes

Timeline: Q2 2026 **Workaround:** Increase Express timeout, works for <10 min assessments

8. TypeScript Strict Mode Disabled (Some Files)

Problem: `// @ts-ignore` used in some places **Impact:** Type safety compromised, bugs sneak through **Fix:** Enable strict mode globally, fix type errors **Timeline:** Q1 2026 **Workaround:** Manual testing catches most issues

---

Future Technical Improvements

1. RAG Architecture (High Priority)

```
// Planned architecture
import { Pinecone } from '@pinecone-database/pinecone';
import OpenAI from 'openai';

const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY });
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

// Index documents
async function indexDocuments(documents: Document[]) {
  const chunks = documents.flatMap(doc =>
    chunkDocument(doc, { size: 1000, overlap: 200 })
  );

  const embeddings = await Promise.all(
    chunks.map(chunk => openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: chunk.text
    }))
  );

  await pinecone.index('bsr-docs').upsert(
    chunks.map((chunk, i) => ({
      id: chunk.id,
      values: embeddings[i].data[0].embedding,
      metadata: { docId: chunk.docId, page: chunk.page, text: chunk.text }
    }))
  );
}
```

```
// Retrieve relevant chunks for requirement
async function retrieveRelevantChunks(requirement: ComplianceRule) {
  const queryEmbedding = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: requirement.title + ' ' + requirement.description
  });

  const results = await pinecone.index('bsr-docs').query({
    vector: queryEmbedding.data[0].embedding,
    topK: 10,
    includeMetadata: true
  });

  return results.matches.map(m => m.metadata.text);
}
```

Benefits:

- Handle unlimited document size
- Reduce token usage by 80%
- Improve accuracy (more relevant context)
- Enable "search similar past assessments" feature

2. Streaming LLM Responses

```
// Instead of waiting for full response
const response = await anthropic.messages.create({ ... });

// Stream tokens as they arrive
const stream = await anthropic.messages.stream({ ... });

for await (const chunk of stream) {
  if (chunk.type === 'content_block_delta') {
    // Send partial response to frontend
    socket.emit('assessment_progress', {
      ruleId: rule.id,
      partialResult: chunk.delta.text
    });
  }
}
```

Benefits:

- Show real-time progress to user

- Perceived performance improvement
- Early feedback (user sees something happening)

3. Multi-Tenancy (Production Required)

```
model Organization {
  id    String @id
  name  String
  users User[]
  packs Pack[]
}

model User {
  id     String @id
  email  String @unique
  orgId  String
  org    Organization @relation(fields: [orgId], references: [id])
}

model Pack {
  id    String @id
  orgId String
  org   Organization @relation(fields: [orgId], references: [id])
}
```

Row-Level Security:

```
CREATE POLICY org_isolation ON packs
  FOR ALL
  USING (org_id = current_setting('app.org_id')::uuid);
```

4. API for Integrations

```
// RESTful API
app.post('/api/v1/assessments', authenticate, async (req, res) => {
  const assessment = await runAssessment(req.body.documents);
  res.json(assessment);
});

// Webhooks
app.post('/api/v1/webhooks', authenticate, async (req, res) => {
  await registerWebhook(req.user.id, req.body.url, req.body.events);
  res.json({ success: true });
```

```
});

// When assessment completes
await fetch(webhook.url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    event: 'assessment.completed',
    data: { assessmentId, status, results }
  })
});
```

5. Batch Processing

```
// Upload 100 projects at once
POST /api/v1/batch-assessments
{
  "assessments": [
    { "projectId": "proj-1", "documents": [...] },
    { "projectId": "proj-2", "documents": [...] },
    ...
  ]
}

// Process in parallel workers
await Promise.all(
  assessments.map(a => assessmentQueue.add('assess', a))
);

// Return batch ID for tracking
{ "batchId": "batch-123", "status": "processing" }
```

Conclusion: Technical Architecture Summary

What We Built:

- Hybrid rules engine (deterministic) + LLM (probabilistic)
- Document processing pipeline (PDF → text → structured assessment)
- Multi-format report generation (PDF via Puppeteer, Excel via ExcelJS)
- Full-stack TypeScript app (React + Express + Prisma + PostgreSQL)

What Works Well:

- Fast iteration (TypeScript + hot reload)
- Type-safe database access (Prisma)
- Reliable PDF generation (Puppeteer)
- Structured LLM outputs (temperature=0 + JSON schema)

What Needs Improvement:

- RAG for large documents (currently truncating)
- Parallel LLM calls (currently serial)
- Caching (currently none)
- Authentication (currently open)
- Testing (currently manual)

Technical Moat:

- 55 proprietary compliance rules (domain expertise)
- Custom LLM prompts optimized for building safety
- Evidence validation logic (anti-hallucination)
- End-to-end workflow (not just document analysis)

Next Technical Milestones:

- Q1 2026: Parallel LLM + caching + authentication
- Q2 2026: RAG implementation + job queue + monitoring
- Q3 2026: Multi-tenancy + API + batch processing
- Q4 2026: ML model fine-tuning on assessment data

Last Updated: March 2026 For questions, contact engineering team